Selling the nines

Baseten already engineers four nines for the world's most demanding AI products. It sells three — and delivers the fourth by hand, one support ticket at a time. This document explains how reliability becomes the product, and why the company that turns scaling, placement, failover, and rollout into declared policy sets the market standard.

THE THESIS

Every inference platform rents the same GPUs, and the routing and disaggregation beneath them are becoming open standards. What cannot be copied is Baseten's operating position: MCM already runs active-active across 20+ clouds, placing and healing workloads competitors would page a human for. Today that machinery is invisible — placement constraints hand-encoded per customer, failover unobservable, the contract a 99.9% shadow of the 99.99% reality. **The winner of production inference turns that machinery inside out: reliability that customers declare as policy, watch enforced in a console, and let agents operate. Baseten is one product surface away.**

VIKRAM SIWACH

JULY 2026

COMPANIONS: DUE-DILIGENCE BRIEF · RELIABILITY PRD · OPERATE CONSOLE

ALL CLAIMS SOURCED §9

01 Why this, why now

Once a model is deployed, production inference is won or lost on keeping it fast, reliable, and economical at scale. For Baseten's customers — Cursor, Notion, OpenEvidence, Abridge, Clay, Gamma, Writer — tokens are cost-of-goods-sold and downtime is revenue interruption, not inconvenience. Reliability is the reason they pay a platform instead of renting GPUs.

Baseten just raised a $1.5B Series F at a $13B valuation — its fourth raise in eighteen months — on the back of 20× revenue growth, 40× inference-volume growth, and more than a billion calls a day. That growth is the strategic problem this document answers: **the reliability that produced it is delivered white-glove.** Region and provider constraints are "manually encoded for each customer" (Baseten's own MCM engineering blog). Regional environments "require initial configuration by Baseten — contact support." Capacity preference is a sales conversation. A motion that works at dozens of enterprise accounts arithmetically cannot work at thousands.

Three external clocks make the productization urgent rather than optional:

> **The engine layer is commoditizing upward.** NVIDIA Dynamo 1.0 shipped disaggregated serving and KV-aware routing as production open source; the Kubernetes Gateway API Inference Extension is standardizing inference-aware routing. When the engine is table stakes, durable differentiation moves up-stack — to placement, capacity, and reliability policy.

> **GPU scarcity is structural, which makes capacity preference monetizable.** On-demand rental capacity is sold out across virtually all GPU types; advanced packaging is allocated through mid-2027. The scarce thing is no longer "a GPU" — it is the right GPU, in the right region, inside the right compliance envelope, at the right moment. Whoever owns the policy for that owns the buying surface.

Compliance is hardening on a fixed date. EU AI Act high-risk obligations become fully applicable August 2, 2026. Baseten's marquee vertical is healthcare — Abridge, OpenEvidence — where residency and auditable egress are procurement gates. "Compliance-bound workloads given right-of-way on sensitive capacity" is not a feature idea; it is what these customers will shortly be legally unable to buy without.

02 The game: what the market prices, and the winner's profile

2.1 · The market buys outcomes but prices promises

Baseten's marketing claims 99.99% uptime for mission-critical workloads; a customer case study claims 100%. The published SLA commits to 99.9% monthly, excludes throttling and queueing on non-reserved capacity, and explicitly does not measure the ability to scale beyond provisioned capacity. The engineering over-delivers the contract by an order of magnitude. **Unpriced engineering is margin left on the table — and a fourth nine that cannot be bought is a fourth nine a competitor can eventually sell first.**

2.2 · Reliability is now a scheduling problem

The classical reliability toolkit — redundant replicas, health checks, retries — assumed capacity was purchasable. Under structural scarcity, reliability degenerates into scheduling: which workload gets the H100s in the EU cluster when there aren't enough for everyone. Priority, preemption, and right-of-way are the reliability primitives of this era, and they only exist where one operator sees all the demand across all the clouds. MCM is that operator. Nobody else has both the vantage point and the customer base that needs it.

2.3 · The winner's profile

OpenAI-compatible APIs make switching trivial, so the durable winner needs advantages competitors cannot copy by adopting the same open source. Baseten's has three layers:

LAYER	THE ADVANTAGE	WHY IT CAN'T BE COPIED
Operating position `MCM`	Active-active across 20+ clouds, already placing and healing at >1B calls/day. The operational telemetry of every failure mode across every provider compounds daily.	Years of multi-cloud incident data and scheduler hardening. Single-cloud players (CoreWeave, Modal) structurally cannot see cross-provider failure; hyperscalers won't route around themselves.
The policy surface `this strategy`	First mover turns operations into APIs: placement, failover, release, and SLO as declared objects. AWS won the mid-2000s by exactly this motion — the JD's own analogy.	Requires the layer above: policy without an enforcement plane is YAML theater. Bedrock's inference profiles — the nearest primitive — are coarse selection, not policy, and NVIDIA-cloud-only economics.
Proof machinery `evidence`	Measured MTTR, drill reports, placement records, per-incident evidence artifacts customers can hand to their own auditors.	Cultural, not technical: publishing measured reliability evidence is a one-way door competitors with weaker numbers cannot walk through.

The honest gap, stated plainly: the thesis's least-proven claim is that customers want to *declare* rather than *delegate*. The answer is defaults-with-override — every policy object ships pre-filled with what Baseten's operators would have done anyway, made visible. Three design partners gate GA on exactly this question, and the console is judged on weekly active use, not shipped features.

03 What we learned first-hand

Every load-bearing belief in this strategy was tested against Baseten's shipped product — six dedicated-deploy attempts, chaos drills against live pools, a working incident agent, and a 19-entry friction log where every entry carries committed evidence (per-request CSVs, deployment IDs, timestamps). The findings, in plain language:

> **The engine over-delivers the contract.** MCM's active-active behavior is real and observable from outside. The SLA sells a third of a nine less than the marketing claims and scopes it to provisioned capacity. This is a packaging problem, not an engineering problem — the best kind to find.

> **Placement is a ticket, not a policy.** Constraints hand-encoded per customer; regional environments via support; org-level GPU gating discoverable only after packaging and uploading a truss. There is no API, console surface, or config object through which a customer can express where their workload may run.

> **The release engine stops at promotion.** Rolling deployments exist but are disabled by default, exist only during a promotion, and do not support Chains. There is no standing canary, no shadow/mirror primitive, no A/B, no metric-gated auto-rollback. Health probes default to 30 minutes — a stuck replica can take traffic for half an hour of someone's product being down.

> **The reliability loop is agent-shaped, and the agent works.** An SLO-driven incident agent — quarantine, streaming-TTFT probes, verified reinstatement, closed action allowlist — took cross-deployment MTTR from *unbounded* (the agent-off control never recovered) to **8.8–9.2 seconds, measured and replayable**, using only Baseten's public surfaces. What's missing is the production-grade warrant.

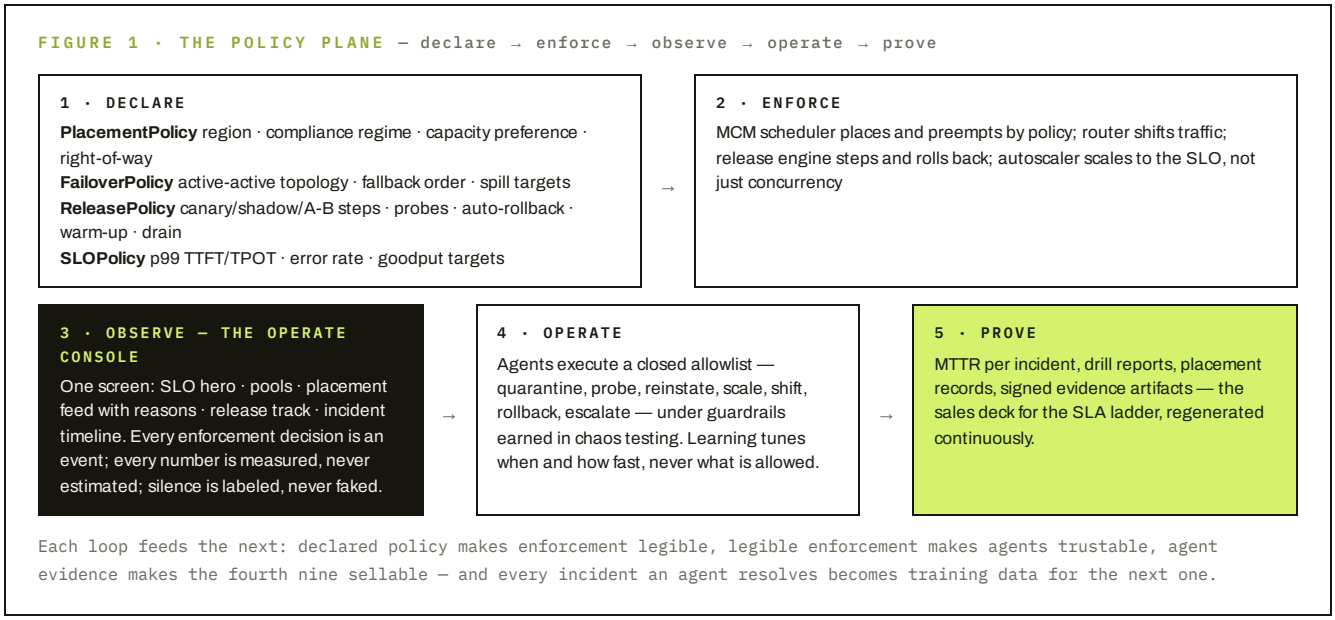
> **The agentic onboarding path already works — Baseten built it.** The same workspace that fought six manual deploy attempts onboarded cleanly through Baseten's own MCP server and agent skills (shipped June 30): attach, baseline, deploy, read metrics — all agent-driven. The friction log and the MCP path are two endings to the same first hour, and the difference between them is the product this strategy productizes for every evaluating customer.

> **The observability contract cannot yet carry a console.** Deployment metrics lag 1–3 minutes and arrive as silent nulls, indistinguishable from "never served." The metrics endpoint 429s a single dashboard user on ~14 sequential GETs. BUILD_FAILED exposes zero API-visible logs. 429s carry no Retry-After. A reliability product must fix its own foundations first — which is why proof is Phase 0, not a slide.

> **The first hour is the weakest hour.** A first deploy sat 30 minutes in silent scheduling limbo, then died as INACTIVE with no reason. A fixed 16 GiB host-RAM SKU OOM-crash-looped an 8B model with no pre-flight warning. One opt-in config stanza (BDN weight caching) cut cold start 360s → 148s — and nothing at push time suggested it. The fast path exists; it is not the default path.

04 The product thesis: the policy plane

Turn MCM inside out. Four declared objects, one console, agents with a warrant:



4.1 · Placement as policy, scarcity as right-of-way

A PlacementPolicy expresses in one object what today takes a support thread: allowed regions, compliance regime (HIPAA, EU-residency), provider preference, and capacity tier (reserved-first, on-demand, spill). Priority classes give compliance-bound workloads right-of-way on sensitive capacity: when EU H100s run short, region-flexible workloads drain and relocate automatically, visibly, with a preemption event in both parties' feeds. Scarcity stops being an outage cause and becomes a scheduling product — and reserved capacity gains a self-serve buying surface it has never had.

4.2 · The release engine beneath safe rollouts

Traffic-shifting becomes a first-class engine rather than a promotion side effect: standing canary splits with metric gates and automatic rollback; shadow/mirror with diff reports (zero user risk — the pattern that de-risks every migration); A/B with cohort stickiness; declarative warm-up and drain so zero-downtime is a property of the platform, not of customer diligence; probe presets per engine replacing the 30-minute defaults. Chains — the compound-AI product — reaches parity instead of being excluded.

4.3 · Agents with a warrant

The MTTR result (§3) proves the loop; the product question is authority. Agents operate under a closed allowlist with hard guardrails (never quarantine the last healthy pool; sticky quarantine that health flaps cannot lift; escalate once, then slow-poll), every action logged to the incident timeline, and reinforcement learning tuning only cadence and thresholds — never expanding the action set. This is how a reliability agent earns its way into production: bounded authority, total visibility, measured MTTR as its scoreboard.

05 Selling the nines: the SLA ladder

The commercial expression of the policy plane is an SLA ladder in which customers earn the nines they configure for — converting engineering that already exists into pricing power:

TIER	WHAT THE CUSTOMER DECLARES	WHAT BASETEN COMMITS
Managed `today's default`	Nothing — white-glove behavior as today, now visible read-only in the operate console.	99.9% as currently contracted. The console alone moves retention: seeing MCM absorb failures is the product's best demo of itself.
Declared	SLOPolicy + ReleasePolicy + PlacementPolicy with reserved capacity floor.	99.95%, with capacity terms inside the SLA for reserved pools — closing the exclusion that makes today's SLA capacity-scoped.
Operated	Multi-region FailoverPolicy + agent remediation enabled + scheduled drills.	99.99% — the number marketing already claims, now bought, with drill reports and per-incident evidence artifacts as the receipt. Billing distinguishes cold-start minutes from serving minutes.

5.1 · Go-to-market: Prove, Declare, Operate — in strict order

MOTION	TARGET & OFFER	WHY THIS ORDER
1 · Prove `now`	Fix the observability contract (failure reasons, capacity states, metrics completeness, rate-limit headers) and ship the read-only operate console to every dedicated account. Three design partners, at least one healthcare.	A reliability product standing on silent nulls and un-triageable failures would be marketing. Phase 0 is a blocking gate: no policy GA until the console's own data sources are contractual. It also fixes the trust-destroying first hour, which compounds every funnel metric.
2 · Declare `next 2 quarters`	Policy objects GA — SLO and Release first (self-contained, immediately valuable), Placement and Failover with design partners. Declared tier launches with reserved capacity attached.	Release and SLO policies win the daily-active habit; Placement converts the compliance deadline (Aug 2026) into demand; each declared policy deepens switching costs because policy accumulates like configuration never does.
3 · Operate `gated on 1–2`	Agent remediation GA, incident console with MTTR, scheduled drills, Operated tier at 99.99%.	Agents are only trustable against declared policy (they need to know what "healthy" means) and only sellable with the evidence machinery running. The MTTR scoreboard becomes the category's public benchmark — set the standard, then publish it.

> **The discipline:** motion 2 does not scale until Phase 0's gates are demonstrably met; motion 3 does not GA until three design partners run declared policy in production for a quarter. Each phase funds the next only after its exit gate — a failed hypothesis costs one phase, not the company.

5.2 · Monitoring is the wedge, migration is the motion

Cursor-class buyers already run across Baseten, RunPod, Modal, and hyperscaler endpoints — multi-homing is how serious inference owners hedge capacity and price. But asking them to adopt a Baseten-owned *control plane* over those providers on day one triggers justified suspicion: nobody hands placement authority to the vendor with the most to gain from the placement. The trust ladder must start lower. **The plane lands as read-only, cross-provider monitoring** — bring your endpoints, get one console for per-route SLO adherence and measured $/Mtok across every provider you run on. No control handed over, nothing to migrate, no lock-in to evaluate: a monitoring tool teams shipping models weekly already need and cannot get in one place today. And onboarding is one prompt, not a form — the customer's own coding agent attaches endpoints and

baselines routes through **Baseten's own shipped rails, the MCP server and agent skills** (June 30, 2026): credit where due, Baseten made the platform agent-operable before this plane existed; the plane is the onboarding those rails deserve.

Monitoring quietly accumulates the asset nobody else has: **the route ledger** — per-route workload shape, SLO adherence, and cost across providers. MCM sees Baseten's supply; the ledger sees the customer's demand, including the share Baseten is losing and exactly why. When the evidence shows a route would hold its SLO at lower measured $/Mtok on Baseten capacity, the console says so: *"this route at equal SLO, 23% cheaper on reserved capacity — shadow it now"* — one click into certified migration (shadow → certify → promote, reversible for the life of the route). **This is a migration play, not a platform ask:** the customer acts at the moment they were already considering moving traffic, with evidence in hand instead of a bake-off project. Declared policies, agents, and right-of-way apply to workloads once they run on Baseten — the control plane is the reward that deepens after traffic arrives, never the day-one ask. Observe → migrate → declare → operate.

The friction log is this motion's map. An evaluating customer's first hour on Baseten today is documented there first-hand — the silent capacity wait, the host-RAM wall, the production pointer left on a dead deploy — and every one of those moments is a migration lost before the evidence could speak. **Phase 0's contract fixes are not DX polish; they are the migration funnel's front door.** The same telemetry that makes the console honest is what makes an evaluator's shadow test conclusive.

> **The suspicion objection answers itself — if the discipline holds.** Monitor-only means monitor-only: no control action is ever offered on an external deployment; the only affordance is certified migration. Win-back cards come only from rerunnable measured evidence; the customer owns and can export their ledger; external legs pass through at zero markup — we monetize serving and the SLA ladder, never other people's tokens. One gamed recommendation kills the plane; honest ones compound into the strongest acquisition channel an inference platform can own.

06 The six commitments

PILLAR	COMMITMENT	PRD MAPPING
P1 · Proof before promise	The observability contract ships first: machine-readable failure reasons on every terminal state, capacity-wait as first-class status, metrics completeness timestamps, published rate-limit budgets with Retry-After. No reliability claim ships on a surface that can silently lie.	Phase 0 · F0.1–F0.5
P2 · Placement as policy	One declared object for region, compliance regime, provider, and capacity preference — self-serve, enforced by MCM, observable in a placement feed with reasons. Right-of-way for compliance-bound workloads on sensitive capacity, with visible preemption events.	Phase 1 · F1.3–F1.4
P3 · Failover as policy	Multi-region / active-active topologies and fallback order declared per workload, drilled on schedule, evidenced per event — never a support anecdote again.	Phase 1–2 · F1.5, F2.2
P4 · The release engine	Standing canary/shadow/A-B with metric gates and auto-rollback; declarative warm-up, drain, and probe presets; Chains parity; fast health defaults.	Phase 1 · F1.2
P5 · MTTR as the headline	Self-serve incident management: detection from declared SLOs, agent remediation under a closed allowlist, incident timelines, per-incident evidence — MTTR measured, published, and owned as the category benchmark.	Phase 2 · F2.1
P6 · Sell the nines	The SLA ladder binds commitments to declared policy and reserved capacity; billing distinguishes cold-start from serving minutes; the 99.99% the marketing claims becomes a product with a price.	Phase 2 · F2.3

07 What we explicitly do not believe

That reliability is a dashboard business. The console is how policy becomes legible; policy is how capacity gets bought. Panels don't retain customers — accumulated declared policy does.

That the SLA gap is a legal problem. It is a packaging problem. Tie the nines to declared policy and reserved capacity and the exclusions become tiers instead of asterisks.

That more knobs mean more reliability. Every policy ships pre-filled with what Baseten's operators would do anyway. Declaration is defaults made visible and overridable — not a config dump.

That agents should own production unsupervised. Closed allowlists, human-gated escalation, learning that tunes cadence but never authority. The 9-second MTTR was achieved *with* guardrails, not despite them.

That self-serve placement cannibalizes enterprise. Capacity preference is a buying surface: every PlacementPolicy with a reserved-first clause is a reserved-capacity order form.

That this can wait. Bedrock is productizing residency-aware routing from above; Dynamo and GAIE are commoditizing routing from below. The window where policy is a differentiator, not a checkbox, is open now.

08 Principal risks

RISK	MITIGATION
Customers delegate, don't declare	Defaults-with-override; console read-only first (value before asks); three design partners gate every policy GA; weekly-active-use is the kill metric, honestly read.
MCM can't cleanly expose per-customer intent	Engineering discovery is Phase 0 work alongside the observability contract; policy objects version from day one; nothing GA's before the scheduler honors it end-to-end in a drill.
99.99% is legally hard under real scarcity	The Operated tier requires multi-region policy plus reserved floors — the SLA prices the configuration that makes it achievable. Drills produce the actuarial evidence before the contract ships.
Self-serve erodes white-glove margin	White-glove doesn't scale past the current account count anyway; the ladder moves revenue from services-shaped to SLA-shaped, which is where the multiple is.
Evidence base is n=1	The friction log is one workspace on a non-enterprise tier; some walls may not exist for enterprise accounts. Phase 0 instruments the real funnel and replaces anecdote with telemetry before Phase 1 spends.
An agent takes a wrong action at scale	Closed allowlist, never-orphan guards, sticky quarantine, escalate-once — then shadow-mode agents (recommend, don't act) as the GA on-ramp; full incident replay for every action.

THE ONE-LINE STRATEGY

Prove the telemetry, let customers declare the reliability they're already getting, put agents on the pager with a warrant, sell the fourth nine the engineering already delivers — and monitor the clouds customers already run on, so traffic migrates to Baseten on measured evidence instead of sales calls.

09 Evidence base — key sources

Baseten Series F announcement, June 22 2026 (baseten.co/blog; businesswire.com) · MCM engineering deep-dive, "How we built Multi-cloud Capacity Management" (baseten.co/blog) · MCM product page — 99.99%, active-active, 20+ clouds (baseten.co/products) ·

Service Level Agreement — 99.9%, capacity-scoped exclusions (baseten.co/service-level-agreement) · docs.baseten.co: autoscaling/overview & traffic-patterns (cron pre-warm guidance), rolling-deployments (promotion-scoped, default-off, no Chains), health-checks (30-min defaults), regional-environments ("contact support"), observability index (no alerting surface; export limited 6 req/min) · Changelog: MCP server + agent skill (Jun 30 2026), max_scale_down_rate (Jun 29 2026) · NVIDIA Dynamo 1.0 production release (developer.nvidia.com) · Kubernetes Gateway API Inference Extension (docs.nvidia.com/dynamo) · SemiAnalysis, GPU rental capacity sold out, March 2026 · EU AI Act high-risk applicability Aug 2 2026 (digital-strategy.ec.europa.eu) · AWS Bedrock cross-region inference profiles (docs.aws.amazon.com) · Modal GPU memory snapshots (modal.com/blog) · First-hand: friction log #1–#19 with committed CSVs, incident-agent MTTR 8.8–9.2s measured vs unbounded control (github.com/vsiwach/baseten-mvp: docs/FRICTION_LOG.md, benchmarks/raw/, learning/) · Role definition: Product Manager, Inference Platform (jobs.ashbyhq.com/baseten).